

Graphics functions in C

There are so many built in graphics functions defined in C library. The basic graphics functions are described as below. Graphics functions are available in two modes-

- i) Text mode graphics functions
- ii) Graphics mode graphics functions

Text mode graphics functions:

Text mode is a personal computer display setting that divides the display screen into **25 rows and 80 columns** in order to display text without images.

In text mode, each box can contain one character. Text mode contrasts with graphics mode, which features an array of pixels instead of text boxes.

Text mode initially came to prominence in the early 1970s, when video display text terminals started to replace teleprinters in human-computer interfacing. This simple display mode dominated the computer's visual interface for much of that decade.

Although text mode is obviously a very limited display method, it is beneficial in regard to its low memory consumption and fast screen manipulation.

The text mode graphics functions place the text in certain area of the screen. Those functions are included in the header file `<conio.h>`. Some of those text mode graphics functions are:

1. **window(int x1,int y1, int x2, int y2);**

ex: `window(10,10, 80,25);`

where, co-ordinate (10,10) represents the upper left corner of displaying window and (80,25) is the lower right corner of the display window.

- 2. **clrscr():** It is used to clear the screen and locates the curser at the beginning of the screen.
- 3. **gotoxy(x,y):** it moves the curser at specified co-ordinates (x,y) position.
- 4. **cputs(string):** it writes a string given to function to the user defined window screen.
- 5. **putch(char) :** it writes a character given to function to the user defined area of window.

6. **textbackground(n)** : It is used to set the background colour in the text mode, where n is an integer referring the background color mode.

Graphical mode functions:

There are so many graphical functions in C graphics library. All the graphics mode functions are in <graphics.h> file. So we must include the graphics.h file to use those functions. To run a c program in graphical mode the most important functions are as below.

1. **initgraph()**: it is one of the function that is used to initialize the computer in graphics mode. Its syntax is as

initgraph(&gdriver, &gmode, "graphics driver path");

2. **closegraph()**: It is the graphical function that is used to close the graphics mode initialized by the initgraph() function.
3. **graphresult()**: it is a graphical function that returns the value 0 if the graphics is detected correctly and the driver is correctly initialized to correct graphics mode other wise it returns some error code than 0.
4. **grapherrormsg(errorcode)**: This function returns the message string corresponding to the errorcode returned by the graphresult() function.
5. **cleardevice()**: This function is used to clear the screen in graphical mode as clrscr() in text mode.

After correctly initialized in graphics mode, we can use the different graphical functions available in the c- library in graphics.h. Following are some basic graphical functions for drawing geometrical objects.

6. **setcolor(color)**: It is a function that is used to set the color of the drawing object with given color. The color is indicated by the integer from 0 to 15 (16 color)
7. **moveto(x, y)**: it is used to move the cursor in display screen at specified co-ordinates by the value (x,y).
8. **outtext(" text string ")**: prints the text string in the current position of screen.
9. **outtextxy(x, y, "string")**: prints the string from the co-ordinate (x,y) position.

10. **lineto(x, y):** draws the line from current position to (x,y) position.
11. **putpixel(x, y, color):** displays a pixel(point) at (x,y) with given color.
12. **line(a, b, c, d):** draws line from (a,b) to (c,d).
13. **circle(x, y, r):** prints circle with center (x,y) and radius r.
14. **rectangle(a, b, c, d):** prints rectangle where (a,b) is upper left coordinate and (c,d) is lower right co-ordinates of rectangle.

Let us revisit the **initgraph()** function.

Graphics mode Initialization – initgraph() function;

First of all we call the **initgraph()** function that will initialize the graphics mode on the computer. The method initgraph() has the following prototype.

void initgraph(int *graphdriver, int *graphmode, char *pathtodriver);

The method initgraph() initializes the graphics system by loading the graphics driver from disk (or validating a registered driver) then putting the system into graphics mode. The method initgraph() also resets all graphics settings (color, palette, current position, viewport, etc.) to their defaults. The initialization result is set to 0 which can be retrieved by calling graphresult().

The initgraph() method has the following parameters.

1) *graphdriver

This is an integer value that specifies the graphics driver to be used. You can give Graphdriver a value using a constant of the graphics_drivers enumeration type which is listed in graphics.h .

Normally we use value as “0” (requests auto-detect). Other values are 1 to 10 and description of each enumeration type is listed here.

graphics_drivers constant	Numeric value
DETECT	0 (requests autodetect)
CGA	1
MCGA	2
EGA	3
EGA64	4
EGAMONO	5

IBM8514	6
HERCMONO	7
ATT400	8
VGA	9
PC3270	10

2) *graphmode

This is an integer value that specifies the initial graphics mode

(unless *graphdriver = DETECT method sets *graphmode to the highest resolution available for the detected graphics driver. You can give *graphmode a value using a constant of the graphics_modes enumeration type and description of each enumeration type is listed here.

Graphics			Columns	
Driver	graphics_mode	Value	x Rows	Palette
CGA	CGAC0	0	320 x 200	C0
	CGAC1	1	320 x 200	C1
	CGAC2	2	320 x 200	C2
	CGAC3	3	320 x 200	C3
	CGAHI	4	640 x 200	2 color
MCGA	MCGAC0	0	320 x 200	C0
	MCGAC1	1	320 x 200	C1
	MCGAC2	2	320 x 200	C2
	MCGAC3	3	320 x 200	C3
	MCGAMED	4	640 x 200	2 color
	MCGAHI	5	640 x 480	2 color
EGA	EGALO	0	640 x 200	16 color
	EGAHI	1	640 x 350	16 color
EGA64	EGA64LO	0	640 x 200	16 color
	EGA64HI	1	640 x 350	4 color
EGA-MONO	EGAMONOH1	3	640 x 350	2 color
	EGAMONOH1	3	640 x 350	2 color
HERC	HERCMONOH1	0	720 x 348	2 color

ATT400	ATT400C0	0	320 x 200	C0
	ATT400C1	1	320 x 200	C1
	ATT400C2	2	320 x 200	C2
	ATT400C3	3	320 x 200	C3
	ATT400MED	4	640 x 200	2 color
	ATT400HI	5	640 x 400	2 color
VGA	VGALO	0	640 x 200	16 color
	VGAMED	1	640 x 350	16 color
	VGAHI	2	640 x 480	16 color
PC3270	PC3270HI	0	720 x 350	2 color
IBM8514	IBM8514HI	0	640 x 480	256 color
	IBM8514LO	0	1024 x 768	256 color



As the drivers are getting advanced, resolution improved.

Note: The main reason to call detectgraph directly is to override the graphics mode that detectgraph recommends to initgraph.

3) *pathtodriver

Specifies the directory path where initgraph() looks for graphics drivers (*.BGI) first.

If they're not there, initgraph() method looks in the current directory.

If pathtodriver is null, the driver files must be in the current directory.

Both *graphdriver and *graphmode parameters must be set to valid graphics_drivers and graphics_mode values or you'll get unpredictable results. (The exception is graphdriver = DETECT.)

```
/* Sample program to draw a line & circle*/  
#include<graphics.h>  
#include<conio.h>  
main()  
{  
    int gd=DETECT,gm;  
    initgraph(&gd,&gm,""); /* initialization of graphic mode */  
    line(10,10,200,200);  
    circle(150,150,100);  
    getch();  
    closegraph(); /* Restore original screen mode */  
}  
/* End of program */
```

For more details click on the following link:

<https://pdfslide.net/reader/f/graphics-functions-in-c>